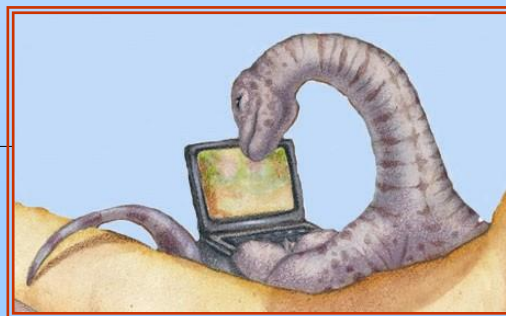


Chapter 1: Introduction





Chapter 1: Introduction

- What Operating Systems Do
- Computer-System Organization
- Computer-System Architecture
- Operating-System Structure
- Operating-System Operations
- Process Management
- Memory Management
- Storage Management
- Protection and Security
- Distributed Systems
- Special-Purpose Systems
- Computing Environments





Objectives

- To provide a grand tour of the major operating systems components
- To provide coverage of basic computer system organization





What is an Operating System?

- A program that acts as an intermediary between a user of a computer and the computer hardware.
 - Coordination between users/programs and the HW.
 - Control.
- Operating system goals:
 - Execute user programs and make solving user problems easier.
 - Make the computer system convenient to use.
 - Use the computer hardware in an efficient manner.





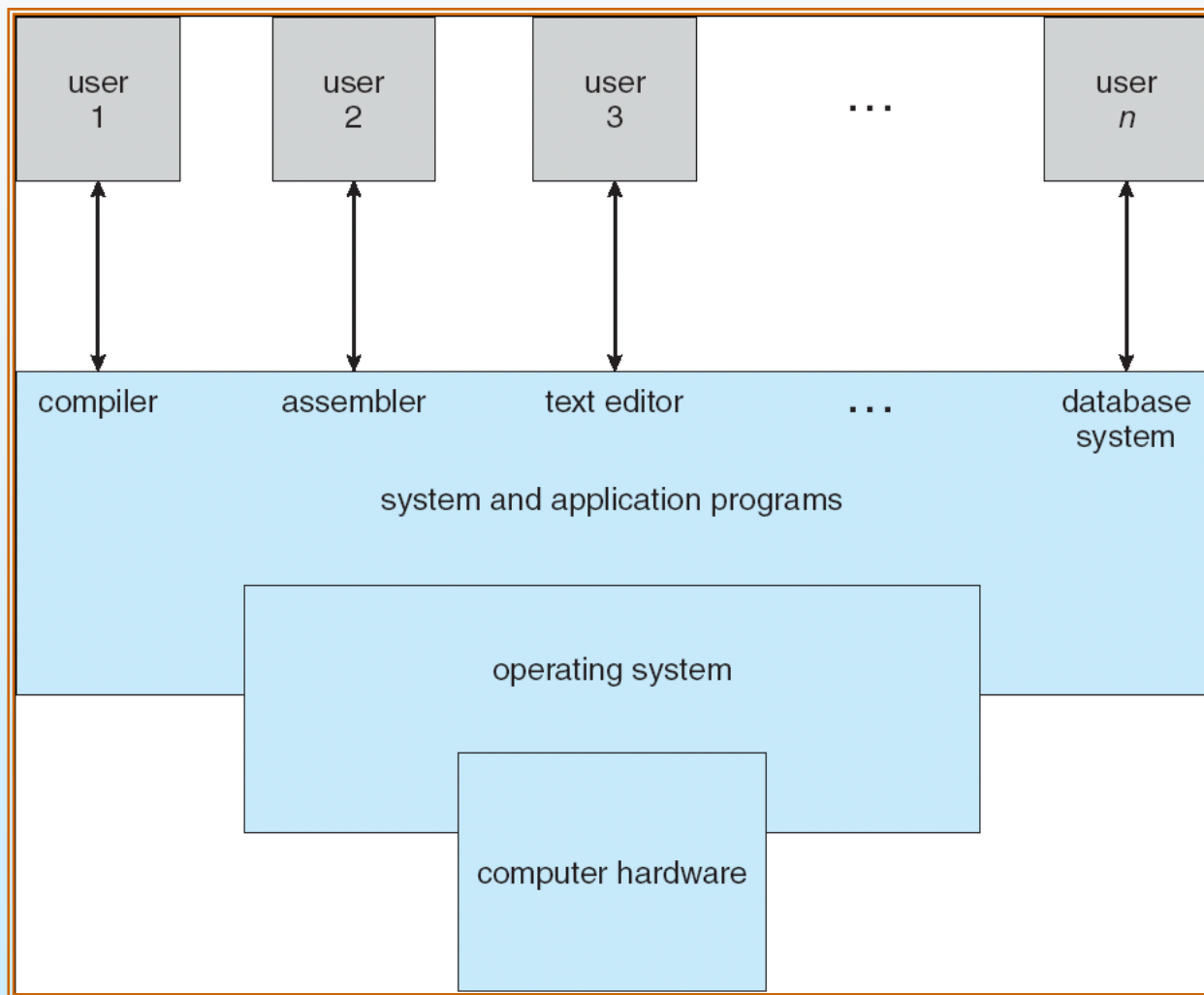
Computer System Structure

- Computer system can be divided into four components
 - Hardware – provides basic computing resources
 - ▶ CPU, memory, I/O devices
 - Operating system
 - ▶ Controls and coordinates use of hardware among various applications and users
 - Application programs – define the ways in which the system resources are used to solve the computing problems of the users
 - ▶ Word processors, compilers, web browsers, database systems, video games
 - Users
 - ▶ People, machines, other computers





Four Components of a Computer System





Operating System Definition

- OS is a **resource allocator**
 - Manages all resources
 - Decides between conflicting requests for efficient and fair resource use
- OS is a **control program**
 - Controls execution of programs to prevent errors and improper use of the computer





Operating System Definition (Cont.)

- “The one program running at all times on the computer” is the **kernel**. Everything else is either a system program (ships with the operating system) or an application program





Computer Startup

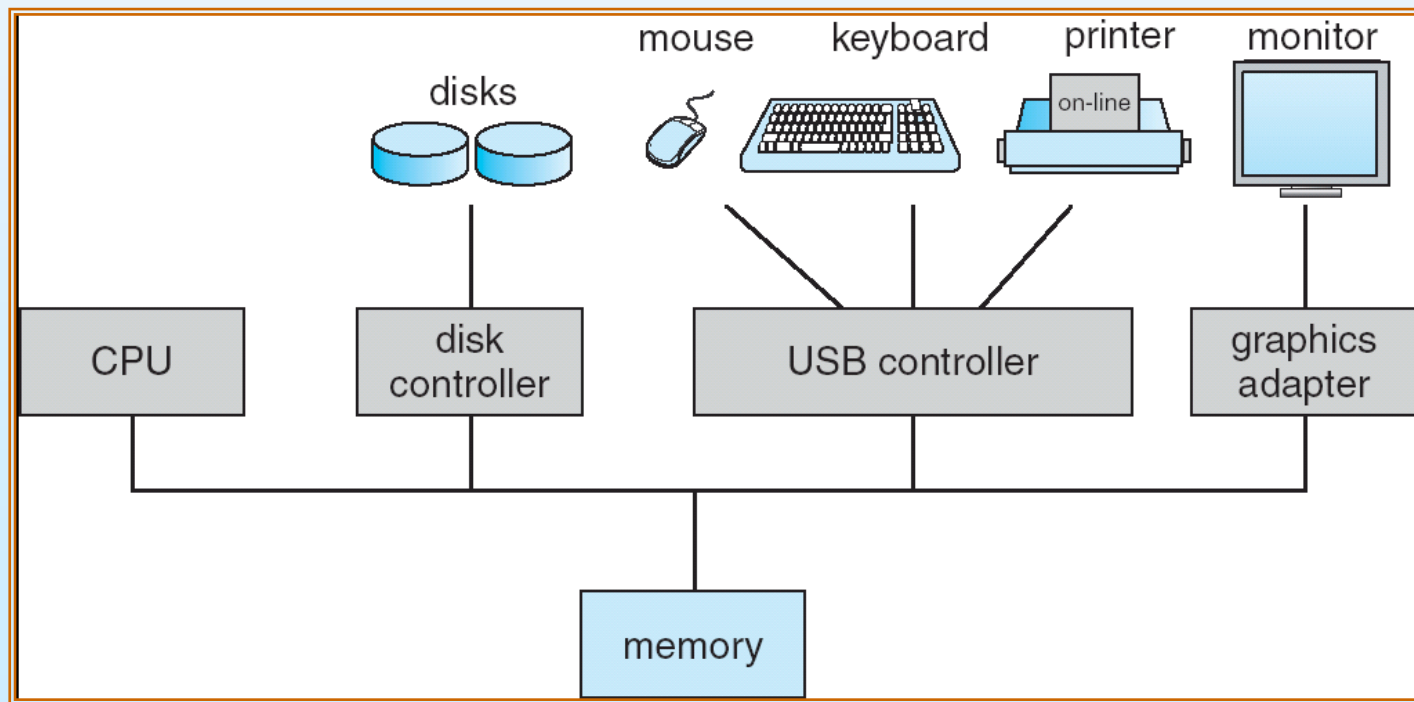
- **bootstrap program** is loaded at power-up or reboot
 - Typically stored in ROM or EEPROM (electrically erasable programmable read-only memory), generally known as **Firmware** : is a type of software that is etched directly into a piece of hardware
 - Initializes all aspects of system
 - Loads operating system kernel and starts execution





Computer System Organization

- Computer-system operation
 - One or more CPUs, device controllers connect through common bus providing access to shared memory
 - Concurrent execution of CPUs and devices competing for memory cycles





Computer-System Operation

- I/O devices and the CPU can execute concurrently (see the figure in slide 12).
- Each device controller is in charge of a particular device type.
- Each device controller has a **local buffer**.
- CPU moves data from/to main memory to/from local buffers
- I/O is from the device to local buffer of controller.
- Device controller informs CPU that it has finished its operation by causing an **interrupt**.
- Any event occurrence is usually signaled **by an interrupt** from the hardware or the software.
 - **Hardware** may trigger an interrupt at any time by **sending a signal** to the CPU through the system bus.
 - **Software** may trigger interrupt by executing a special operation called **system call (monitor call)**.





Common Functions of Interrupts

- Interrupt transfers control to the interrupt service routine generally, through the *interrupt vector*, which contains the addresses of all the service routines, those addresses for all services are loaded by the OS at the OS initialization phase.
 - We have the interrupt vector (imagine it as an array) which contains the addresses of all the service routines, when the CPU is interrupted it stops what it is doing and immediately transfer execution to a fixed location, this location is the starting address where the service routine for the interrupt is stored.
- Interrupt architecture must save the address of the interrupted instruction.
 - When the CPU is interrupted it stop what is doing and start to execute the new interrupt, when it finishes the execution for the current interrupt it should return to the previous interrupted computation by retrieving its address (so this address should be saved).
- Incoming interrupts are *disabled* while another interrupt is being processed to prevent a *lost interrupt*.
- A *trap (exception)* is a software-generated interrupt caused either by an error (ex: division by zero or invalid memory access) or a user program request. We need a trap software to issue interrupts to the CPU when needed (ex: division by zero).
- An operating system is *interrupt driven*.





Interrupt Handling

- The operating system preserves the state of the CPU by storing **registers** and **the program counter**, so when the CPU is interrupted we should save its current information about registers and the program counter.
- Two important definitions:
- Both spooling/pooling needs a buffer to save jobs.
 - **Spooling**: used for output
 - ▶ Ex: saving a number of jobs (for a printer as an example) in a buffer because the printer can not print all the jobs at the same time).
 - **Pooling**: used for input.





- Each device has a **controller which maintains local buffer** and special purpose registers.
- The device controller is responsible for **moving data** between the peripheral devices that it controls and its local buffer storage.
- OS have a device driver for each device controller to understand the device controller and presents a uniform interface to the device to the rest of the operating system.
- To start an I/O operation:
 - the driver loads the appropriate registers within the device controller (ex: loads instructions that mean: “read a specific data from hard disk”).
 - The controller examines the contents for these registers to determine what action to take (ex: the action is: transfer a data).
 - Then the controller starts the transfer of data from the device to its local buffer. (at this stage we have two options to continue this transfer of data, we will see them in the next slide)





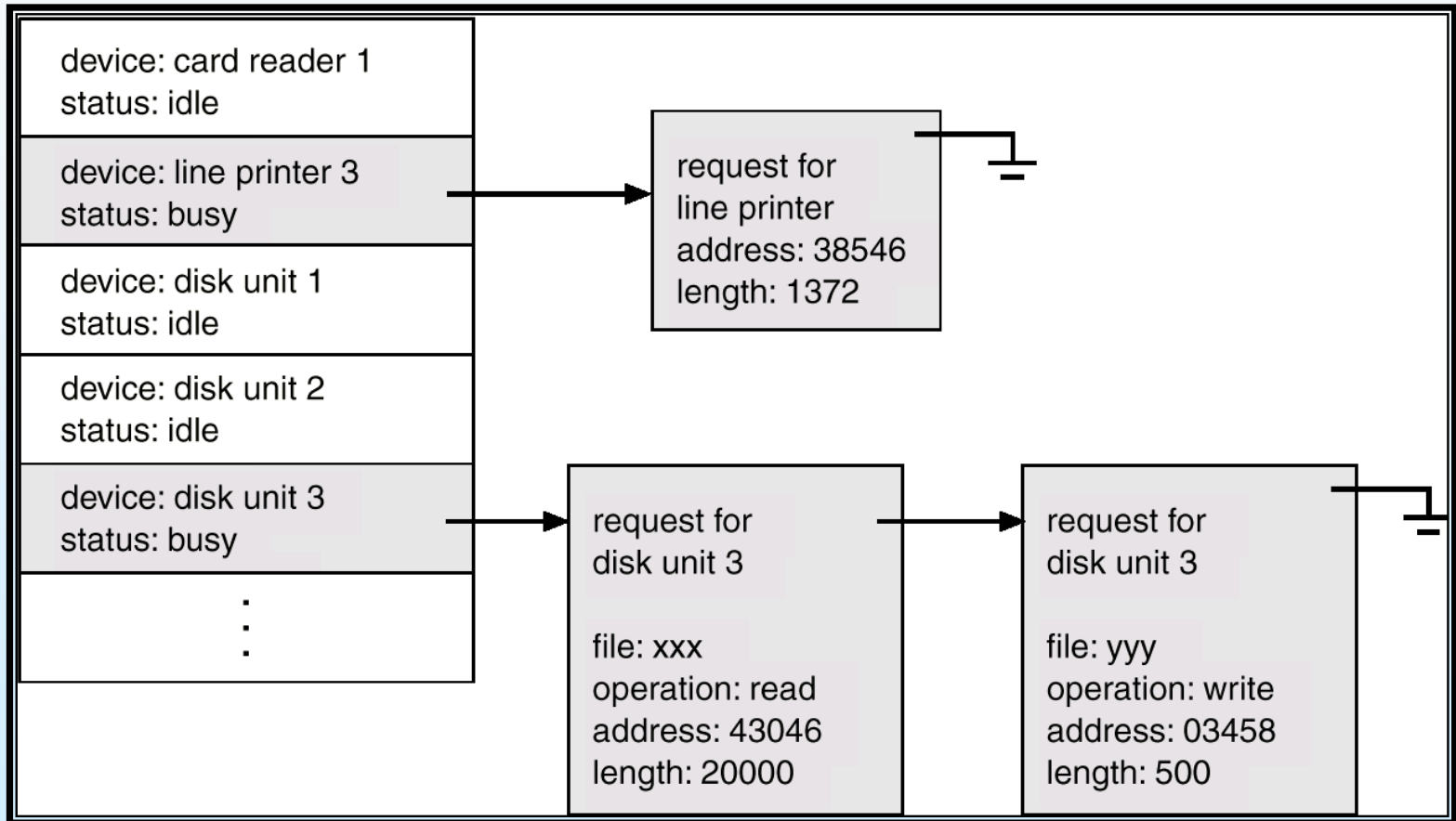
I/O Structure

- 1) After I/O starts, control returns to user program only upon I/O completion. (suitable for small amounts of data).
 - Wait instruction idles the CPU until the next interrupt
 - Wait loop (contention for memory access). the CPU waits until the data transfer completed and the device controller inform the device driver that it is finished form transferring data to the device buffer.
 - At most one I/O request is outstanding at a time, no simultaneous I/O processing.
- 2) After I/O starts, control returns to user program without waiting for I/O completion. (suitable for large amounts of data (bulk data)).
 - *System call* – request to the operating system to allow user to wait for I/O completion and the CPU execute other tasks until it is interrupted again by the device driver which have been tolled by the device controller that it have finished its I/O job.
 - In this case the device controller use direct memory access (DMA) to perform its job while the CPU execute other job.
 - *Device-status table* contains entry for each I/O device indicating its type, address, and state.
 - Operating system indexes into I/O device table to determine device status and to modify table entry to include interrupt.





Device-Status Table





Direct Memory Access (DMA) Structure

- Used for high-speed I/O devices able to transmit information at close to memory speeds.
- Device controller transfers blocks of data from buffer storage directly to main memory without CPU intervention.
- During the transference of data the CPU is free to other jobs.
- Only one interrupt is generated per block to inform the CPU about the data which have been moved so far, rather than the one interrupt per byte.





Storage Structure

- Main memory – only large storage media that the CPU can access directly.
- Secondary storage – extension of main memory that provides large nonvolatile storage capacity.
 - Magnetic disks – rigid metal or glass platters covered with magnetic recording material
 - ▶ Disk surface is logically divided into *tracks*, which are subdivided into *sectors*.
 - ▶ The *disk controller* determines the logical interaction between the device and the computer.





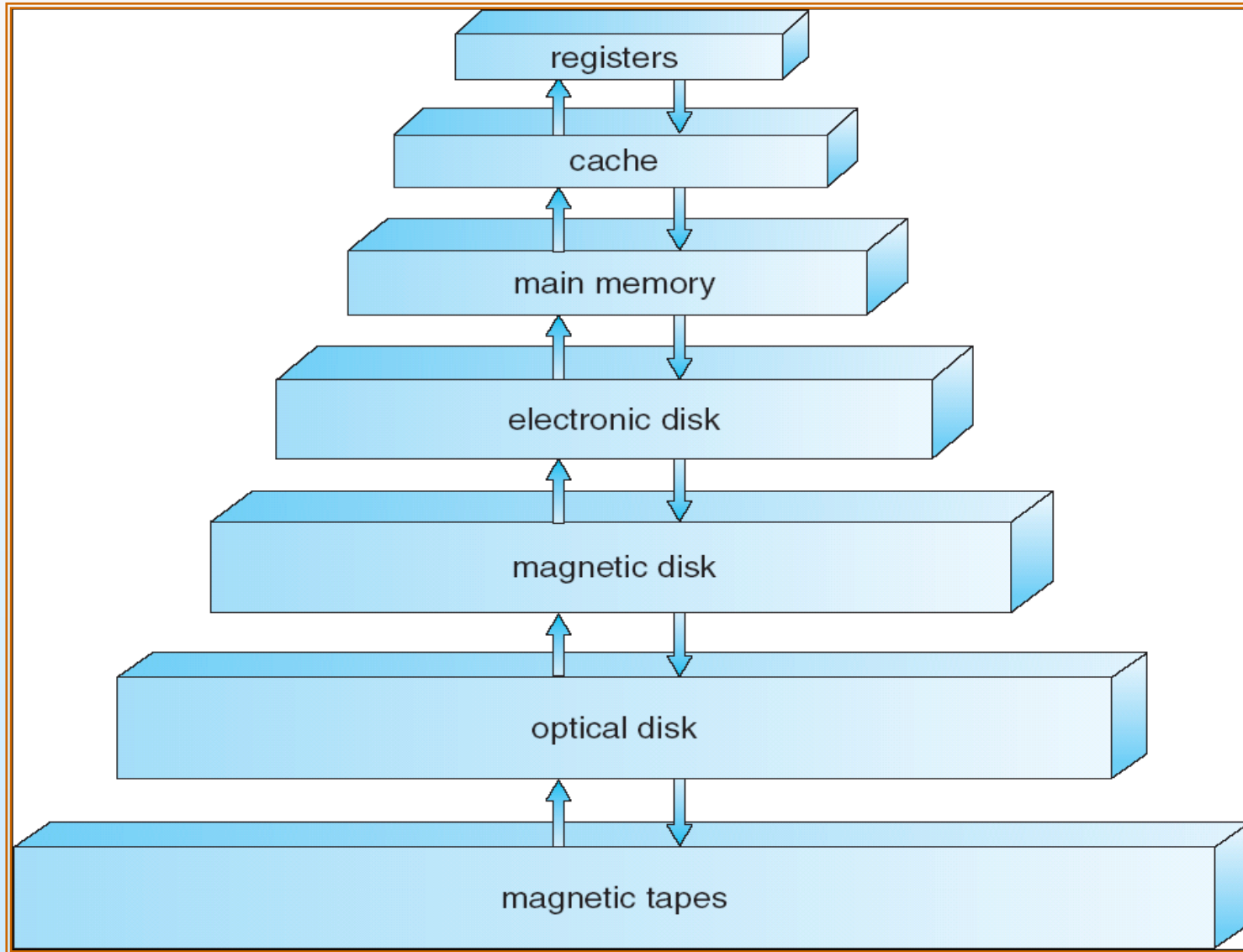
Storage Hierarchy

- Storage systems organized in hierarchy according to:
 - Speed
 - Cost
 - Volatility
- *Caching* – copying information into faster storage system; main memory can be viewed as a last *cache* for secondary storage.





Storage-Device Hierarchy





Caching

- Important principle, performed at many levels in a computer (in hardware, operating system, software)
- Information in use copied from slower to faster storage temporarily
- Faster storage (cache) checked first to determine if information is there
 - If it is, information used directly from the cache (fast)
 - If not, data copied to cache and used there
- Cache smaller than storage being cached so we need:
 - Cache management important design problem
 - Cache size and replacement policy





Performance of Various Levels of Storage

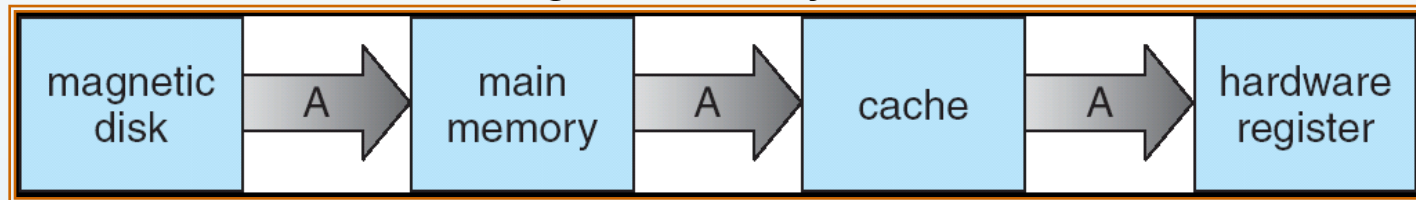
Level	1	2	3	4
Name	registers	cache	main memory	disk storage
Typical size	< 1 KB	> 16 MB	> 16 GB	> 100 GB
Implementation technology	custom memory with multiple ports, CMOS	on-chip or off-chip CMOS SRAM	CMOS DRAM	magnetic disk
Access time (ns)	0.25 – 0.5	0.5 – 25	80 – 250	5,000.000
Bandwidth (MB/sec)	20,000 – 100,000	5000 – 10,000	1000 – 5000	20 – 150
Managed by	compiler	hardware	operating system	operating system
Backed by	cache	main memory	disk	CD or tape





Migration of Integer A from Disk to Register

■ Multitasking environments must be careful to use most recent value (because we could have more than one copy for a certain data, ex: variable x can be changed at cache while it is still not changed at hard disk), no matter where it is stored in the storage hierarchy



■ Multiprocessor environment must provide **cache coherency** (it is a problem in multiprocessor and in distributed environment; it can be solved by updating) in hardware such that all CPUs have the most recent value in their cache (CPU must sure that an update to the value in one cache reflected in all other caches)

■ Distributed environment situation even more complex

- Several copies of a datum can exist in distributed locations, we must ensure that all locations have the same value for a given shared data, so if this shared data modified in one location; the other locations should be updated as soon as possible (the locations are connected using network (what if it is disconnected?)).





Terminologies

- A program is loaded into memory and executing it is called process.
- When a process execute , it typically executes for a short time before it either finishes or needs to perform I/O.
- Job pool : it is on hard disk, it contains the process ready and awaiting for allocation to the main memory.
- Job scheduling: we need it to choose which processes in the pool we should bring to main memory.
 - ▶ we can not bring all the jobs to the MM because of the size limitations.
- CPU scheduling: we need it if several jobs in the main memory are ready to execute at the same time.





Operating System Structure

- **Multiprogramming** (it is main idea is efficiency in using CPU, memory, and peripheral devices) (see point 5).
 - 1) Single user cannot keep CPU and I/O devices busy at all times
 - 2) Multiprogramming organizes jobs (code and data) so CPU always has one to execute
 - 3) A subset of total jobs (total jobs are kept in a place called jobs pool) in system is kept in memory simultaneously (see slide 28)
 - 4) One job from them is selected (from the pool) and run via **job scheduling**
 - 5) When it has to wait (for I/O for example), OS switches to another job (**using CPU scheduling**) until the first job finishes its I/O, then the OS return the control to it.





Operating System Structure

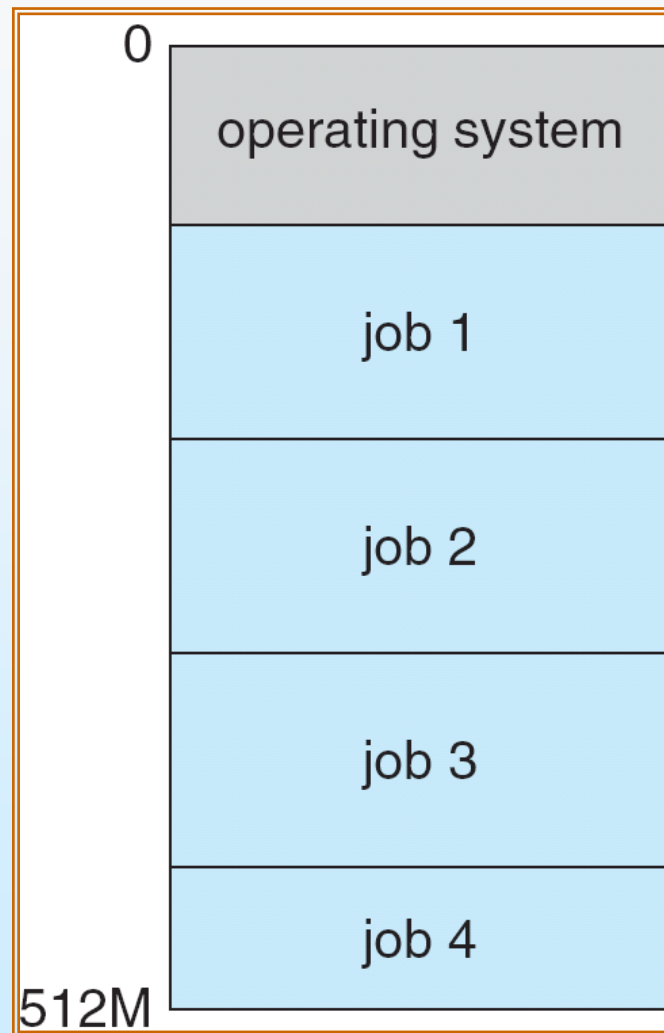
■ **Timesharing (multitasking)** (its main idea is for user interaction, to run more than one program at the same time or more than one user can use the same computer) is logical extension of multiprogramming in which CPU switches jobs so frequently that users can interact with each job while it is running, creating **interactive** computing

- **Response time** should be < 1 second
- Each user has at least one program executing in memory $\Rightarrow$ **process**
- If several jobs ready to run at the same time $\Rightarrow$ **CPU scheduling**
- If processes don't fit in memory (i.e. the memory is small), **swapping** moves them in (MM) and out (hard disk) to run
- If processes don't fit in memory (i.e. the memory is small), another technique can be used called **Virtual memory (chapter 9)**, VM allows execution of processes not completely in memory
- A timeshared OS uses CPU scheduling and job scheduling to provide the users with a small portion of a time shared computer.





Memory Layout for Multiprogrammed System





Operating-System Operations

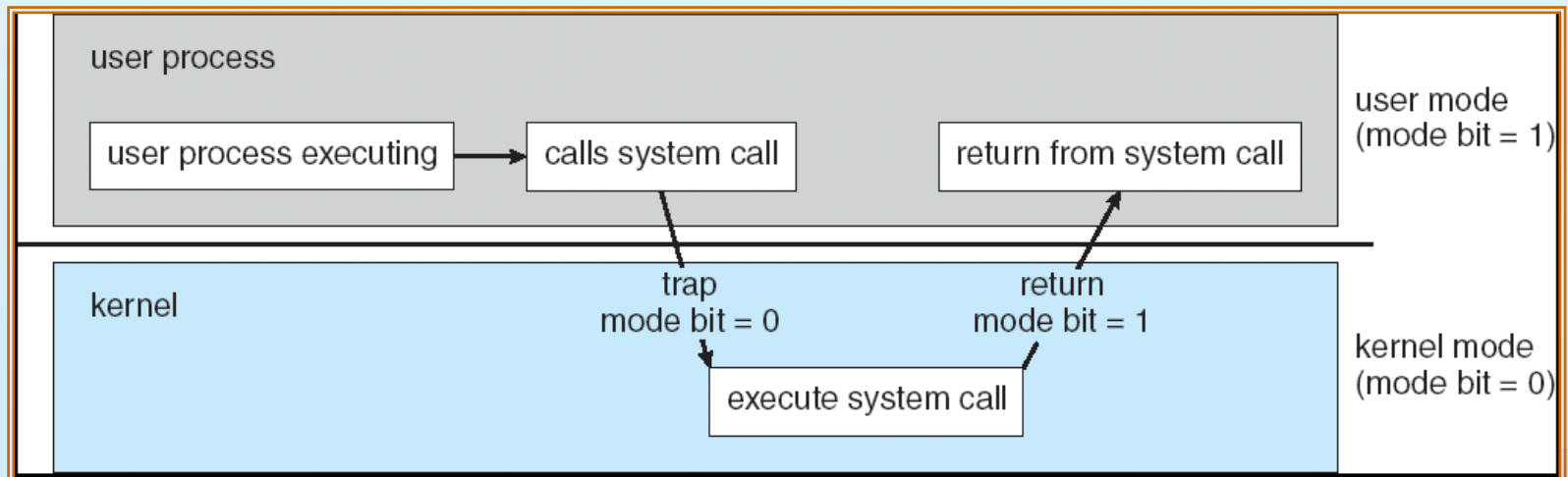
- Modern OS systems are interrupt driven by hardware.
- Software error or request creates **exception** or **trap** (like interrupt but generated by SW error or request. without this (trap generated SW) the OS will not be able to serve more than one process (ex: enter an infinite loop without a tool to stop it (i.e. trap generated SW))).
 - Division by zero
 - request for operating system service
- Other process problems include infinite loop, processes modifying each other or the operating system
- **Dual-mode** operation allows OS to protect itself and other system components
 - **User mode** and **kernel mode (also called supervisor mode, privileged mode, and system mode)** are used to distinguish between the OS code and the user code.
 - **Mode bit** (0 for kernel, and 1 for user) provided by hardware
 - ▶ Provides ability to distinguish when system is running user code or kernel code
 - ▶ Some instructions designated as **privileged**, only executable in kernel mode
 - ▶ System call, trap, and interrupt changes mode to kernel.
 - ▶ System call is issued when a user application requests a service from the OS.





Transition from User to Kernel Mode

- To ensure that OS maintain control over CPU, the OS should have:
- Timer to prevent infinite loop / process hogging resources
 - Set interrupt after specific period
 - Operating system decrements counter until it reaches zero.
 - When counter is zero generate an interrupt
 - we set up the (time period) before we schedule the process to regain control or terminate program that exceeds allocated time





Process Management

- A process is a program in execution. It is a unit of work within the system. Program is a *passive entity*, process is an *active entity*.
- Process needs resources to accomplish its task
 - CPU, memory, I/O, files
 - Initialization data
- Process termination requires reclaim (return back) of any reusable resources
- Single-threaded (chapter 4) process has one **program counter** specifying location of next instruction to execute
 - Process executes instructions sequentially, one at a time, until completion
- Multi-threaded process has one program counter per thread
- Typically system has many processes, some user, some operating system running concurrently on one or more CPUs
 - Concurrency by multiplexing the CPUs among the processes / threads





Process Management Activities

The operating system is responsible for the following activities in connection with process management:

- Creating and deleting both user and system processes
- Suspending and resuming processes
- Providing mechanisms for process synchronization
- Providing mechanisms for process communication
- Providing mechanisms for deadlock handling





Memory (Main Memory) Management

- Memory in computer is an array of bits. To deal with the memory (bits) the most important thing is to know the addresses for those bits.
- All data will be in memory before and after processing
- All instructions will be in memory in order to execute
- CPU can access the MM only, so to reach a data on the disk it should loaded first to MM.
- Memory management determines what is in memory when
Optimizing CPU utilization and computer response to users
- Memory management activities
 - Keeping track of which parts of memory are currently being used and by whom
 - Deciding which processes (or parts thereof) and data to move into and out of memory
 - Allocating and deallocating memory space as needed





Storage Management

■ To make computer more convenient for users, OS provides uniform, logical view of information storage

- Abstracts physical (the actual way the data stored on a disk) properties to logical (abstract way for easy understand and deal with) storage unit such as **file**
- Each physical medium (CD, Disk, Etc..) is controlled and accessed by a storage device (i.e., disk drive, tape drive)
 - ▶ Varying properties for those physical mediums include access speed, capacity, data-transfer rate, access method (sequential or random)

■ File-System management

- Files usually organized into directories
- Access control on most systems to determine who can access what
- OS activities include
 - ▶ Creating and deleting files and directories
 - ▶ Primitives to manipulate files and dirs
 - ▶ Mapping files onto secondary storage
 - ▶ Backup files onto stable (non-volatile) storage media





Mass-Storage (physical medium) Management

- Usually disks used to store data that does not fit in main memory or data that must be kept for a “long” period of time.
- Proper management is of central importance
- Entire speed of computer operation hinges on disk subsystem and its algorithms
- OS activities
 - Free-space management
 - Storage allocation
 - Disk scheduling (when a number of files ordered to be loaded at the same time from the hard disk, we need disk scheduling)
- Some storage need not be fast
 - Tertiary storage includes optical storage, magnetic tape, CD, DVD
 - Still must be managed
 - Tertiary medias varies between WORM (write-once, read-many-times) and RW (read-write)





I/O Subsystem

- One purpose of OS is to hide peculiarities (properties) of hardware devices from the user
- I/O subsystem responsible for
 - Memory management of I/O including buffering (storing data temporarily while it is being transferred), caching (storing parts of data in faster storage for performance), spooling (used for output. For example: a printer with 5 jobs at the same time).
 - General device-driver interface
 - Drivers for specific hardware devices





Protection and Security

- If the computer has multiple users, and allows the concurrent execution of multiple processes, then access to data must be regulated. For that purpose, mechanisms ensure that files, memory, CPU, and other resources can be operated on by only those processes that have gained proper authorization from the OS. So protection is :
 - **Protection** – any mechanism for controlling access of processes or users to resources defined by the OS. (ex: dual mode protection, 0/1: user/system mode).





Protection and Security

- A system can have adequate protection but still be prone to failure and allow inappropriate access (stolen authentication information), so we need security.
 - **Security** – defense of the system against internal and external attacks
 - ▶ Huge range, including denial-of-service, worms, viruses, identity theft, theft of service
 - Systems generally first distinguish among users, to determine who can do what, this can be done by:
 - ▶ User identities (**user IDs**, security IDs) include name and associated number, one per user
 - ▶ User ID then associated with all files, processes of that user to determine access control
 - ▶ Group identifier (**group ID**) allows set of users to be defined and controls managed, then also associated with each process, file
 - ▶ **Privilege escalation (raising)** allows user to change to effective ID with **more rights**





Computing Environments

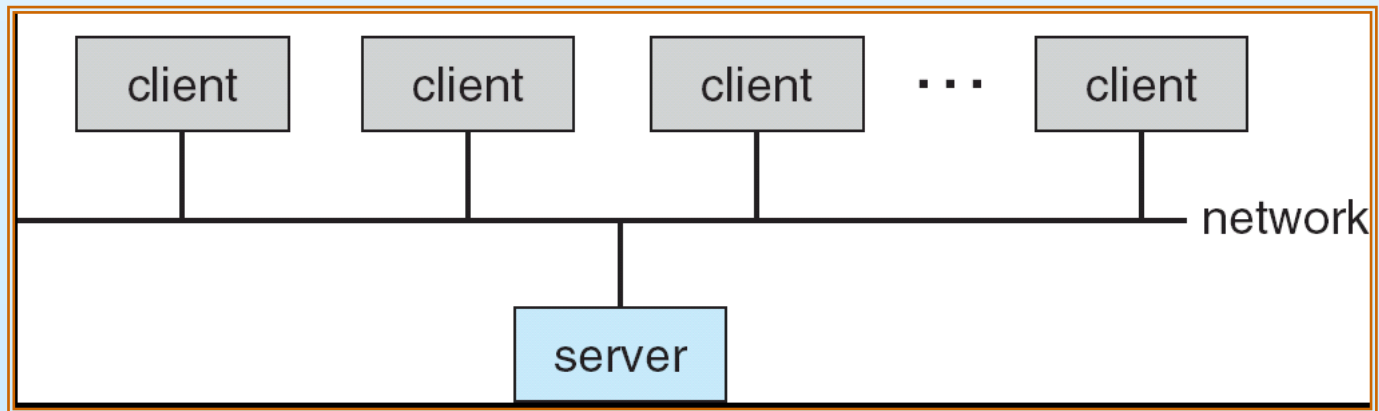
- So far, we have provided an overview of computer-system organization and major OS components. Now we introduce a brief overview of how these are used in a variety of computing environments.
- Traditional computer
 - Office environment
 - ▶ PCs connected to a network, terminals attached to mainframe or minicomputers providing timesharing
 - ▶ Now portals allowing networked and remote systems access to same resources
 - Home networks
 - ▶ Used to be single system, then modems
 - ▶ Now firewalled, networked





Computing Environments (Cont.)

- Client-Server Computing
 - Many systems now **servers**, responding to requests generated by **clients**
 - ▶ **Compute-server** provides an interface to client to request services (i.e. database)
 - ▶ **File-server** provides interface for clients to store and retrieve files





Peer-to-Peer Computing

- Another model of distributed system
- P2P does not distinguish clients and servers
 - Instead all nodes are considered peers
 - May each act as client, server or both
 - Node must join P2P network





Web-Based Computing

- Web has become ubiquitous (movable and continues)
- Web computing has increased.
- PCs most prevalent (the mostly used) devices
- More devices becoming networked to allow web access (mobiles)
- New category of devices to manage web traffic among similar servers: **load balancers**



End of Chapter 1

